LAB GUIDE · 32 LABS · AWS / AZURE / GCP

Cloud Computing Across Three Providers

Hands-on exercises for every video in the course. Each concept is practiced once as how it works, once as how it fails, and always with an eye on the bill.

How to use this guide

Do the labs in order. Labs 1–2 need no accounts; Lab 3 builds the environment every later lab depends on. Most labs name all three providers — where a lab says “two providers” or “one provider,” pick the ones you want to learn most, and rotate across the course.

Every lab follows the same anatomy:

Objective	What you can do when you finish.
Before you start	Earlier labs, tools, and knowledge you need.
Tasks	Numbered steps, with example CLI commands. Replace anything in <angle brackets>.
Verify	Checks that prove the lab worked. Don't skip these — a lab you can't verify isn't finished.
Teardown	What to delete before you close the terminal.
If it goes wrong	The errors beginners hit most often, and what they usually mean.
Reflect	Two questions to answer in your notebook. They're what turns steps into understanding.

About the commands. They're written for a Linux or macOS shell. On Windows, use WSL — or skip local installs entirely and use each provider's browser-based Cloud Shell, which comes pre-authenticated. A trailing `\` means the command continues on the next line. Lines starting with `#` are comments.

Where a step shows commands for only one or two providers, do the others in the console — the step text tells you what to look for. Every provider can do everything in this guide from its console.

Lab rules

1. **Budget alarms first.** No resource is created on an account without a budget alert (Lab 3).

2. **Tear down the same day.** Leaving resources running is the classic beginner bill. End every session with the teardown audit in Appendix C.
3. **Never use root or global admin for lab work.** Use the admin identity created in Lab 3.
4. **Tag everything.** Follow the convention in Appendix A so you can find what you made.
5. **Lab accounts only.** Security and failure exercises are run in your own isolated lab accounts — never against anything real or anyone else's.
6. **Change one thing at a time.** When something breaks, note what you changed before you change anything else.

Cost key

Estimates assume you tear down within the stated time, in a low-cost region. Prices change — check the provider's pricing calculator before larger labs.

`$0`	No cloud resources, or free tier only.
`< $1`	Small paid resources for under an hour.
`$1–5`	Load balancers, managed databases, NAT gateways for a few hours.
`$5–15`	Managed Kubernetes or multi-cloud labs. Tear down immediately.

Lab index

Lab	Title	Time	Cost
01	Your Everyday Cloud Inventory	30 min	$0
02	Map the Global Footprint	45 min	$0
03	Build Your Three-Cloud Lab Safely	2–3 h	$0
04	Least Privilege in Three Clouds	90 min	$0
05	Write a Diagnostic Plan	45 min	$0
06	Virtual Machines and Auto-Scaling	2 h	< $1
07	Object Storage, Tiering, and the Public Bucket	90 min	< $1
08	Build a Virtual Network	2.5 h	$1–5
09	Load Balancing and Network Peering	2.5 h	$1–5
10	Managed Databases and Recovery	2 h	$1–5
11	Infrastructure as Code, Twice	2 h	< $1
12	Monitoring, Alerts, and Audit Trails	2 h	< $1
13	Managed Kubernetes on Two Providers	3 h	$5–15
14	Event-Triggered Functions	2 h	$0
15	An Event-Driven Flow That Survives Failure	90 min	$0
16	Query a Data Warehouse and Watch the Cost	90 min	< $1
17	Compare Pre-Built AI Services	90 min	< $1
18	Measure the Edge	90 min	< $1
19	Migrate a Workload	3 h	$1–5
20	Design a Well-Architected System	2 h	$0
21	Survive the Loss of a Zone	2.5 h	$1–5
22	Security Posture Assessment	2 h	< $1
23	Detect and Investigate Suspicious Activity	2 h	< $1
24	Find and Remove Waste	2 h	$0
25	Deploy from a Commit	3 h	< $1

Lab	Title	Time	Cost
26	Guardrails at Organization Scale	2 h	$0
27	One App, Three Clouds	3–4 h	$5–15
28	Choose a Provider	90 min	$0
29	Root-Cause a Multi-Layer Fault	2 h	< $1
30	Access, Connectivity, and Configuration Failures	3 h	$1–5
31	Performance and Cost Anomalies	2.5 h	$1–5
32	Your Development Plan	90 min	$0

LAB 01 · Video 1

Your Everyday Cloud Inventory

TIME	EST. COST	PROVIDERS
30 min	$0	None needed

Objective. Recognize cloud services in daily life and classify each by service model and deployment model.

BEFORE YOU START

- A notebook or spreadsheet

TASKS

1. List at least ten online services you used in the last 24 hours — email, file storage, streaming, maps, banking, work chat, and any developer tools.
2. For each, note who actually runs the infrastructure. If unsure, look for the provider in the service's status page, privacy policy, or job listings.
3. Classify each as SaaS, PaaS, or IaaS from your point of view as the user. Write one sentence on where the responsibility line falls.
4. Classify each by deployment model: public, private, hybrid, or multi-cloud (best guess, with your reasoning).
5. Pick three services and mark which of the five characteristics you can observe directly: on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service.
6. For one service, describe what it would cost you — in money, effort, or data — to leave it. That is lock-in.

VERIFY

- ☐ At least ten rows, each with a service model and a deployment model
- ☐ At least one PaaS or IaaS example
- ☐ Every classification has a one-line justification

IF IT GOES WRONG

- Not sure whether something is SaaS or PaaS? Ask: do I write code that runs on it (PaaS) or just use it (SaaS)?
- Big consumer apps are often multi-cloud behind the scenes — “best guess with reasoning” is the goal, not certainty.

Reflect

1. Which of your services would hurt most if its provider had a day-long outage?
2. Where did the responsibility line surprise you?

LAB 02 · Video 2

Map the Global Footprint

TIME	EST. COST	PROVIDERS
45 min	$0	AWS · Azure · GCP

Objective. Compare how the three providers structure regions, zones, and edge locations, and identify the regions nearest you.

BEFORE YOU START

- A web browser

TASKS

1. From each provider's global infrastructure page, record the number of regions, which regions have three or more availability zones, and the number of edge or point-of-presence locations.
2. Note the naming conventions side by side (e.g. us-east-1 / East US / us-east1) and what a zone is called in each.
3. Identify the three regions closest to you on each provider.
4. Measure round-trip latency to those regions using a browser-based region latency test. Run it twice at different times of day.
5. After Lab 3, repeat the region listing from the CLI to confirm your notes:

   ```
   aws ec2 describe-regions --output table
   az account list-locations -o table
   gcloud compute regions list
   ```

6. Choose a "home region" per provider for the rest of the course: low latency, three zones, and broad service availability.

VERIFY

- ☐ A comparison table covering all three providers
- ☐ Latency measured for at least three regions per provider
- ☐ A home region chosen for each, with a reason

IF IT GOES WRONG

- Latency tests vary with Wi-Fi and time of day — run each twice and keep the lower value.
- Newer regions sometimes lack services. Check a service you'll use (e.g. managed Kubernetes) is offered in your home region.

Reflect

1. Why might a company pick a region that isn't the closest?
2. What does a zone protect you from that a region doesn't?

LAB 03 · Video 3

Build Your Three-Cloud Lab Safely

TIME	EST. COST	PROVIDERS
2–3 h	$0	AWS · Azure · GCP

Objective. Create isolated, protected, budget-capped lab accounts on all three providers, with authenticated CLIs and a proven teardown habit.

BEFORE YOU START

- An email address you control that supports aliases
- A credit card (required for sign-up even on free tiers)
- A phone with an authenticator app
- A terminal — or use each provider's browser-based Cloud Shell

TASKS

1. Create a dedicated email alias for each account (e.g. you+aws@...). Sign up for AWS, Azure, and GCP. Read each free-tier page and write down three limits per provider.
2. Before anything else: enable MFA on the root / global administrator account of each. Store recovery codes offline.
3. Create budgets with email alerts at 50%, 80%, and 100% of a low monthly cap (e.g. $10): AWS Budgets, Azure Cost Management budget, GCP Billing budget. Trigger or preview a test notification where the provider allows it.
4. Create a separate administrative identity for daily lab work. You will not sign in as root again except to fix billing or recovery.
5. Install and authenticate the three CLIs, signing in as the daily admin identity:

```
aws configure sso        # or aws configure for an IAM user
az login
gcloud init
```

6. Create an isolated container for lab work on each provider:

```
az group create -n rg-lab03 -l <region> --tags course=cloud3 lab=lab03
gcloud projects create cloud3-lab-<initials>
gcloud config set project cloud3-lab-<initials>
```

7. Adopt the naming and tagging convention in Appendix A.

8. Deploy one free-tier resource on each — a storage bucket:

```
aws s3 mb s3://lab03-<initials>-aws
az storage account create -n lab03<initials> -g rg-lab03 --sku Standard_LRS
gcloud storage buckets create gs://lab03-<initials>-gcp --location=<region>
```

9. Tear down all three and confirm nothing remains:

```
aws s3 rb s3://lab03-<initials>-aws --force
az group delete -n rg-lab03 --yes
gcloud storage rm -r gs://lab03-<initials>-gcp
```

VERIFY

- [ ] MFA is on for every root / global admin account
- [ ] A budget with alerts exists on all three accounts
- [ ] Each CLI returns your non-root identity (aws sts get-caller-identity, az account show, gcloud auth list)
- [ ] Resource listings are empty after teardown

TEARDOWN

- [ ] Delete the three buckets and the lab resource group
- [ ] Keep the accounts, budgets, and CLIs — every later lab uses them

IF IT GOES WRONG

- Budget alerts can take hours to fire and do not stop spending — they only warn you. Teardown is still your job.
- “Command not found” after installing a CLI usually means you need to open a new terminal so your PATH updates.
- Bucket names are global: if creation fails with “already exists,” add more characters to your initials.
- GCP projects need a billing account linked before most APIs work: gcloud billing projects link <project> --billing-account=<id>.

Reflect

1. What is the single biggest risk to your wallet in this lab setup, and what protects you from it?
2. Where would you look first if a charge appeared tomorrow?

LAB 04 · Video 4

Least Privilege in Three Clouds

TIME	EST. COST	PROVIDERS
90 min	$0	AWS · Azure · GCP

Objective. Create a least-privilege identity on each provider and prove the boundary holds by testing what it cannot do.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- Comfort switching CLI profiles (aws --profile, az login, gcloud config configurations)

TASKS

1. Create a test bucket on each provider containing one harmless text file.
2. AWS: create a user (or role) with a policy that allows only s3:GetObject and s3:ListBucket on that one bucket.

```
aws iam create-user --user-name lab04-reader
aws iam put-user-policy --user-name lab04-reader \
  --policy-name read-one-bucket --policy-document file://policy.json
```

3. Azure: create a user in Entra ID and assign Storage Blob Data Reader scoped to the lab resource group only.

```
az role assignment create --assignee <user-id> --role "Storage Blob Data Reader" --
scope <rg-id>
```

4. GCP: create a service account and grant roles/storage.objectViewer on the bucket only.

```
gcloud iam service-accounts create lab04-reader
gcloud storage buckets add-iam-policy-binding gs://lab04-<initials> \
  --member=serviceAccount:lab04-reader@<project>.iam.gserviceaccount.com \
  --role=roles/storage.objectViewer
```

5. As each identity, confirm the allowed action works: read the file.
6. As each identity, attempt three things it should not do: delete the file, list a different bucket, create a VM. Copy each error message exactly.
7. On AWS, add an explicit Deny for s3:GetObject alongside the Allow and observe that deny wins.

VERIFY

- [ ] Allowed read succeeds on all three providers
- [ ] Nine denied attempts recorded with their error text
- [ ] Explicit deny overrides allow on AWS

TEARDOWN

- [ ] Delete the users, service account, role assignments, and any access keys
- [ ] Delete the test buckets

IF IT GOES WRONG

- IAM changes can take a minute to propagate — wait before concluding a policy “didn’t work.”
- If an allowed action fails, check the resource side too: a bucket policy can deny what the identity policy allows.
- Azure data-plane roles (Storage Blob Data Reader) differ from control-plane roles (Reader). Using the wrong one is a classic beginner mistake.

Reflect

1. How did each provider express “no access” by default?
2. What would you give a CI pipeline: a user, a role, or a service account — and why?

LAB 05 · Video 5

Write a Diagnostic Plan

TIME	EST. COST	PROVIDERS
45 min	$0	Provider-neutral

Objective. Apply the define → locate → evidence → test → evaluate method to a described fault, including whose responsibility it is and what it costs.

SCENARIO

A web application on a cloud VM began returning timeouts at 09:10. At 09:05 a teammate changed a firewall rule. The provider's status page shows "degraded performance" for a storage service in a neighboring region. Users report the site is slow, not fully down.

BEFORE YOU START

- Videos 1–5
- Appendix D worksheet

TASKS

1. Define the problem in one precise sentence: who is affected, what they see, since when.
2. List the layers that could be at fault — identity, configuration, network, service, provider — and rank them by likelihood with reasons.
3. Answer the shared-responsibility question: mine (configuration or code) or the provider's? What would change your answer?
4. Name the specific evidence you would gather and where it lives on each provider (activity log, metrics, flow logs, status page).
5. Propose the single change you would test first, and how you would know it worked.
6. Estimate the cost of the fix and the cost of the outage per hour.

VERIFY

- ☐ The first test is one change, not several
- ☐ Every hypothesis names the evidence that would confirm or rule it out

IF IT GOES WRONG

- A change made five minutes before the incident is the strongest lead — but confirm it with evidence before rolling back.
- “Degraded” on a status page in another region rarely explains your fault. Check whether your resources depend on that region.

Reflect

1. What evidence would prove the problem is the provider’s?
2. What would you do differently if the site were fully down instead of slow?

LAB 06 · Video 6

Virtual Machines and Auto-Scaling

TIME	EST. COST	PROVIDERS
2 h	< $1	AWS · Azure · GCP

Objective. Launch, resize, and terminate a VM on each provider, then watch auto-scaling add and remove instances under load.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- An SSH key pair (ssh-keygen -t ed25519)
- Your public IP (curl ifconfig.me)

TASKS

1. Launch the smallest free-eligible Linux VM on each provider, tagged per Appendix A, SSH restricted to your IP.

```
aws ec2 run-instances --image-id <ami-id> --instance-type t3.micro --key-name <key> \
  --tag-specifications "ResourceType=instance,Tags=[{Key=lab,Value=lab06}]"
az vm create -g rg-lab06 -n vm-lab06 --image Ubuntu2204 --size Standard_B1s --
generate-ssh-keys
gcloud compute instances create vm-lab06 --machine-type=e2-micro --zone=<zone> -
-labels=lab=lab06
```

2. SSH in and record CPU and memory:

```
nproc && free -h
```

3. Stop each VM, resize to the next size up, start it, and re-check. Note how long each provider took and whether the public IP changed.
4. Look up the on-demand, committed-use, and spot price for the same size on each. Record the percentage savings.
5. On one provider, create an auto-scaling group (AWS Auto Scaling group, Azure VM Scale Set, or GCP managed instance group): min 1, max 3, target CPU 50%.
6. Drive load on the instance and watch it scale out, then stop and watch it scale in:

```
sudo apt-get install -y stress-ng
stress-ng --cpu 2 --timeout 600s
```

VERIFY

- [ ] Resize completed on all three providers
- [ ] Scaling activity shows at least two instances during load and one afterwards
- [ ] Pricing table shows on-demand vs committed vs spot

TEARDOWN

- [ ] Delete the scaling group and its launch template / instance template first — otherwise it will recreate instances
- [ ] Terminate all VMs; delete leftover disks and public IPs

IF IT GOES WRONG

- “Connection timed out” on SSH: the firewall or security group doesn’t allow port 22 from your current IP.
- “Permission denied (publickey)”: wrong key or wrong username (ubuntu, ec2-user, azureuser, or your local name on GCP).
- Resizing fails if the larger size isn’t available in that zone — try another size or zone.
- Auto-scaling reacts on a delay of several minutes. Be patient before deciding it isn’t working.

Reflect

1. How much would this VM cost for a month on each pricing model?
2. What would make spot instances a bad choice?

LAB 07 · Video 7

Object Storage, Tiering, and the Public Bucket

TIME	EST. COST	PROVIDERS
90 min	< $1	AWS · Azure · GCP

Objective. Store and retrieve objects on each provider, configure lifecycle tiering, and find and fix a misconfigured public bucket.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- A small text file to upload

TASKS

1. Create a bucket / container on each provider. Upload, list, and download a test file; compare checksums.

```
aws s3 cp test.txt s3://lab07-<initials>/
az storage blob upload --account-name lab07<initials> -c data -f test.txt -n test.txt
gcloud storage cp test.txt gs://lab07-<initials>/
sha256sum test.txt downloaded.txt
```

2. Add a lifecycle rule on each: move objects to an infrequent-access tier after 30 days, archive after 90, delete after 365.

```
aws s3api put-bucket-lifecycle-configuration --bucket lab07-<initials> \
  --lifecycle-configuration file://lifecycle.json
az storage account management-policy create --account-name lab07<initials> \
  -g rg-lab07 --policy @policy.json
gcloud storage buckets update gs://lab07-<initials> --lifecycle-file=lifecycle.json
```

3. Compare the per-GB storage and retrieval prices of each tier. Note which tier would be cheapest for a file read once a year.
4. On one provider, deliberately make a bucket public (non-sensitive file only). Confirm anonymous access works:

```
curl -I https://<public-object-url>
```

5. Find the exposure using the provider's tooling: S3 Block Public Access and IAM Access Analyzer, the storage account's anonymous-access setting and Defender recommendations, or GCP public access prevention and Security Command Center.

6. Fix it and re-run the curl.

```
aws s3api put-public-access-block --bucket lab07-<initials> --public-access-block-
configuration \

BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBucket
s=true
az storage account update -n lab07<initials> -g rg-lab07 --allow-blob-public-
access false
gcloud storage buckets update gs://lab07-<initials> --public-access-prevention
```

VERIFY

- [ ] Checksums match after round-trip on all three
- [ ] Lifecycle rules visible on each bucket
- [ ] Anonymous request returns 403 / 404 after the fix

TEARDOWN

- [ ] Empty and delete all buckets and the storage account

IF IT GOES WRONG

- 403 Forbidden as the owner usually means you're signed in as a different identity — run the "who am I" command.
- New AWS buckets block public access by default; you must turn that off deliberately to stage the exposure.
- Lifecycle rules run asynchronously, often once a day — you won't see objects move during the lab.

Reflect

1. Durability vs availability: which did each tier trade away?
2. How would you prevent a public bucket across a whole organization?

LAB 08 · Video 8

Build a Virtual Network

TIME	EST. COST	PROVIDERS
2.5 h	$1–5	AWS · Azure · GCP

Objective. Build a network with public and private subnets on each provider, place resources correctly, and diagnose a deliberate connectivity failure.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- Lab 6 (SSH to VMs)
- Basic CIDR notation

TASKS

1. Plan addressing before building: 10.8.0.0/16 network, 10.8.1.0/24 public, 10.8.2.0/24 private. Use the same plan on all three.
2. Create the VPC / VNet and both subnets on each provider.

```
aws ec2 create-vpc --cidr-block 10.8.0.0/16
az network vnet create -g rg-lab08 -n vnet-lab08 --address-prefix 10.8.0.0/16 \
  --subnet-name public --subnet-prefixes 10.8.1.0/24
gcloud compute networks create vpc-lab08 --subnet-mode=custom
gcloud compute networks subnets create public --network=vpc-lab08 --range=10.8.1.0/24
--region=<region>
```

3. Configure internet access for the public subnet only (internet gateway and route on AWS; public IP on Azure and GCP).

```
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --vpc-id <vpc-id> --internet-gateway-id <igw-id>
aws ec2 create-route --route-table-id <rtb-id> --destination-cidr-block 0.0.0.0/0 --
gateway-id <igw-id>
```

4. Place a web VM in the public subnet and an app VM (no public IP) in the private subnet. Allow SSH to the web VM from your IP only, and to the app VM from the public subnet only.

```
aws ec2 authorize-security-group-ingress --group-id <sg-id> --protocol tcp --port 22 --cidr <your-ip>/32
az network nsg rule create -g rg-lab08 --nsg-name nsg-web -n allow-ssh --priority 100 \
  --source-address-prefixes <your-ip>/32 --destination-port-ranges 22 --access Allow
gcloud compute firewall-rules create allow-ssh --network=vpc-lab08 --allow=tcp:22 --source-ranges=<your-ip>/32
```

5. Confirm: you can reach the web VM from the internet, the app VM from the web VM, and not the app VM from the internet.
6. Optional: add outbound internet for the private subnet with a NAT gateway — it bills hourly, so delete it within the hour.
7. Staged failure: have a partner (or yourself, noting it down) remove one rule or route. Diagnose with AWS Reachability Analyzer, Azure Network Watcher IP flow verify, or GCP Connectivity Tests.

```
az network watcher test-ip-flow -g rg-lab08 --vm vm-app --direction Inbound --protocol TCP \
  --local 10.8.2.4:22 --remote 10.8.1.4:60000
gcloud network-management connectivity-tests create lab08-test \
  --source-instance=<web-vm-uri> --destination-instance=<app-vm-uri> --protocol=TCP --destination-port=22
```

VERIFY

- [ ] Reachability matrix matches the design on all three providers
- [ ] The staged failure is identified by the tool, not by guessing

TEARDOWN

- [ ] Delete VMs, NAT gateways, public IPs, then the networks
- [ ] Check for leftover network interfaces and elastic IPs

IF IT GOES WRONG

- Overlapping address ranges can’t be connected later — plan ranges once and reuse the plan.
- Security groups / NSGs are stateful: allow inbound and the reply is automatic. Network ACLs are not.
- A VM with a public IP but no route to an internet gateway is still unreachable on AWS.
- The NAT gateway is the usual surprise charge in this lab.

Reflect

1. What made a subnet “public” on each provider?
2. Which diagnostic tool gave the clearest answer?

LAB 09 · Video 9

Load Balancing and Network Peering

TIME	EST. COST	PROVIDERS
2.5 h	$1–5	One or more

Objective. Distribute traffic across instances with a load balancer, peer two networks, and diagnose unhealthy targets.

BEFORE YOU START

- Lab 8 network
- Two VMs running a web server

TASKS

1. Launch two web VMs, each serving a page that returns its hostname and a /health endpoint.

```
sudo apt-get install -y nginx
hostname | sudo tee /var/www/html/index.html
echo ok | sudo tee /var/www/html/health
```

2. Create a load balancer (AWS Application Load Balancer, Azure Load Balancer or Application Gateway, GCP HTTP(S) Load Balancer) with a health check on /health.

```
aws elbv2 create-target-group --name tg-lab09 --protocol HTTP --port 80 \
  --vpc-id <vpc-id> --health-check-path /health
aws elbv2 create-load-balancer --name alb-lab09 --subnets <subnet-a> <subnet-b> --
security-groups <sg-id>
aws elbv2 register-targets --target-group-arn <tg-arn> --targets Id=<vm-1> Id=<vm-2>
```

3. Confirm traffic is distributed:

```
for i in $(seq 10); do curl -s http://<lb-address>/; done
```

4. Create a second network with a non-overlapping range (10.9.0.0/16). Peer it with the first and add routes on both sides.

```
aws ec2 create-vpc-peering-connection --vpc-id <vpc-a> --peer-vpc-id <vpc-b>
az network vnet peering create -g rg-lab09 -n a-to-b --vnet-name vnet-a --remote-vnet
vnet-b --allow-vnet-access
gcloud compute networks peerings create a-to-b --network=vpc-a --peer-network=vpc-b
```

5. Ping or curl across the peering. Then try a third network that overlaps — note the error.

6. Staged failure: stop the web server on one VM, or block the health-check port. Watch the target go unhealthy, find the cause from the load balancer's health detail, fix it.

```
aws elbv2 describe-target-health --target-group-arn <tg-arn>
```

VERIFY

- [ ] Responses alternate between both hostnames
- [ ] Cross-network traffic succeeds over peering
- [ ] Unhealthy target diagnosed from health-check output

TEARDOWN

- [ ] Delete the load balancer and target groups / backend pools
- [ ] Delete peering connections, VMs, and the second network

IF IT GOES WRONG

- Unhealthy targets are most often a firewall rule that blocks the load balancer's own health-check source range.
- The health check path must return 200 — a redirect (301) counts as unhealthy on many load balancers.
- Peering isn't transitive: A ↔ B and B ↔ C does not give A ↔ C.

Reflect

1. At which layer did your load balancer operate, and what would change at the other layer?
2. Why can't overlapping networks be peered?

LAB 10 · Video 10

Managed Databases and Recovery

TIME	EST. COST	PROVIDERS
2 h	$1–5	One or more

Objective. Deploy a managed relational and a NoSQL database and prove automated backup and point-in-time recovery work.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- Lab 8 (private networking)
- A PostgreSQL client (psql)

TASKS

1. Create the smallest managed PostgreSQL instance on one provider: Amazon RDS, Azure Database for PostgreSQL Flexible Server, or Cloud SQL. Private access only.

```
aws rds create-db-instance --db-instance-identifier db-lab10 --engine postgres \
  --db-instance-class db.t3.micro --allocated-storage 20 --master-username labadmin \
  --manage-master-user-password --no-publicly-accessible --backup-retention-period 7
az postgres flexible-server create -g rg-lab10 -n db-lab10-<initials> --tier
Burstable \
  --sku-name Standard_B1ms --backup-retention 7
gcloud sql instances create db-lab10 --database-version=POSTGRES_16 --tier=db-f1-
micro \
  --region=<region> --enable-point-in-time-recovery
```

2. Confirm automated backups are on and set retention to 7 days.
3. Create a table, insert 100 rows, and note the exact time. Wait five minutes, then delete 50 rows.
4. Restore to a new instance at the time before the delete. Connect and count rows.

```
aws rds restore-db-instance-to-point-in-time --source-db-instance-identifier db-
lab10 \
  --target-db-instance-identifier db-lab10-restored --restore-time <UTC-timestamp>
az postgres flexible-server restore -g rg-lab10 -n db-lab10-restored \
  --source-server db-lab10-<initials> --restore-time <UTC-timestamp>
gcloud sql instances clone db-lab10 db-lab10-restored --point-in-time=<UTC-timestamp>
SELECT count(*) FROM lab_rows;   -- expect 100
```

5. Note the restored instance's endpoint — it is different. Write down what an application would need to change.
6. Create a NoSQL table on one provider (DynamoDB on-demand, Cosmos DB serverless, or Firestore). Write and read ten items. Enable point-in-time recovery where offered.

```
aws dynamodb create-table --table-name lab10 --billing-mode PAY_PER_REQUEST \
  --attribute-definitions AttributeName=id,AttributeType=S --key-schema
AttributeName=id,KeyType=HASH
aws dynamodb update-continuous-backups --table-name lab10 \
  --point-in-time-recovery-specification PointInTimeRecoveryEnabled=true
```

VERIFY

- [ ] Restored instance contains 100 rows
- [ ] You can name the endpoint change a failover or restore causes
- [ ] NoSQL reads return the items written

TEARDOWN

- [ ] Delete the original and restored database instances (skip final snapshots for lab data)
- [ ] Delete the NoSQL table

```
IF IT GOES WRONG
```

- Can't connect? Check, in order: network path, firewall rule on port 5432, username, SSL requirement.
- Cloud SQL point-in-time recovery must be enabled before the timestamp you restore to.
- Restores create a new instance — they never overwrite the original.

Reflect

1. What did the provider do for you that you'd otherwise script yourself?
2. How would the application find the restored database?

LAB 11 · Video 11

Infrastructure as Code, Twice

TIME	EST. COST	PROVIDERS
2 h	< $1	Two providers

Objective. Define the same infrastructure in code on two providers, detect drift, and prove reproducibility by destroying and redeploying.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- Git installed
- A code editor

TASKS

1. Install Terraform or OpenTofu. Write a configuration that creates a network, a subnet, and a bucket on two providers, all tagged per Appendix A.
2. Initialize, plan, and apply:

```
terraform init
terraform plan -out lab11.plan
terraform apply lab11.plan
```

3. Change a tag on one resource in the console. Run plan again and read the drift it reports.
4. Destroy everything, then apply again from the same code. Compare resource IDs and timings.

```
terraform destroy
terraform apply
```

5. Optional: express one of the resources in the provider-native tool (CloudFormation, Bicep, or Deployment Manager / Infrastructure Manager) and compare the two files.

VERIFY

- ☐ Plan detects the console change as drift
- ☐ Second apply recreates identical infrastructure with no manual steps
- ☐ State file contains no secrets and is not committed to a public repo

TEARDOWN

- [ ] terraform destroy on both providers
- [ ] Delete any remote state bucket once the lab is done

IF IT GOES WRONG

- “Resource already exists”: it was created outside Terraform. Import it or delete it.
- Never edit the state file by hand. If apply half-succeeded, re-run plan — it shows what’s left.
- Run terraform fmt and validate before plan to catch typos early.

Reflect

1. What would a reviewer see in a pull request that they couldn’t see in the console?
2. How would you share state safely with a teammate?

LAB 12 · Video 12

Monitoring, Alerts, and Audit Trails

TIME	EST. COST	PROVIDERS
2 h	< $1	AWS · Azure · GCP

Objective. Configure metrics, an alert, and audit logging on each provider, then use the telemetry to explain a staged problem.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- Lab 6 (VMs)
- Lab 7 (buckets)

TASKS

1. Enable the audit trail: an AWS CloudTrail trail, an Azure Activity Log diagnostic setting to a Log Analytics workspace, and GCP Cloud Audit Logs (including Data Access logs for storage).

```
aws cloudtrail create-trail --name lab12 --s3-bucket-name lab12-<initials>-trail
aws cloudtrail start-logging --name lab12
az monitor log-analytics workspace create -g rg-lab12 -n law-lab12
```

2. Launch a small VM on each and confirm CPU and network metrics arrive in CloudWatch, Azure Monitor, and Cloud Monitoring.
3. Create an alert: CPU above 70% for 5 minutes, notifying your email.

```
aws cloudwatch put-metric-alarm --alarm-name lab12-cpu --namespace AWS/EC2 --metric-
name CPUUtilization \
  --statistic Average --period 300 --evaluation-periods 1 --threshold 70 \
  --comparison-operator GreaterThanThreshold --dimensions Name=InstanceId,Value=
<instance-id> \
  --alarm-actions <sns-topic-arn>
az monitor metrics alert create -g rg-lab12 -n lab12-cpu --scopes <vm-id> \
  --condition "avg Percentage CPU > 70" --window-size 5m --action <action-group-id>
```

4. Build one dashboard per provider with CPU, network, and estimated daily cost.
5. Staged problem: run stress-ng on the VM, and separately delete an object from a bucket.
6. Using only telemetry, answer: when did CPU spike, did the alert fire and how fast, who deleted the object, from which IP, at what time.

```
aws cloudtrail lookup-events --lookup-attributes AttributeKey=Username,AttributeValue=
<user> --max-results 20
az monitor activity-log list --offset 2h -o table
gcloud logging read 'protoPayload.methodName="storage.objects.delete"' --limit=5
```

VERIFY

- [ ] Alert email received on all three providers
- [ ] The deletion is found in the audit trail with identity and timestamp

TEARDOWN

- [ ] Delete VMs, alert rules, and dashboards
- [ ] Delete the Log Analytics workspace and any log sinks — log storage bills by volume
- [ ] Keep the AWS management-events trail if it is the free one

IF IT GOES WRONG

- AWS records object deletions only if S3 data events are enabled on the trail; Azure needs storage diagnostic logs for blob deletes.
- Alerts that never fire: check the evaluation period, the threshold direction, and that the notification channel was confirmed.
- Audit logs can lag by several minutes.

Reflect

1. Which question could the audit trail answer that metrics couldn’t?
2. What would you alert on for a production web app?

LAB 13 · Video 13

Managed Kubernetes on Two Providers

TIME	EST. COST	PROVIDERS
3 h	$5–15	Two providers

Objective. Deploy the same containerized application to managed Kubernetes on two providers and compare the experience.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- Docker installed
- kubectl installed (and eksctl for AWS)
- Check current control-plane pricing

TASKS

1. Build a small web app image and push it to each provider's registry (Amazon ECR, Azure Container Registry, Artifact Registry).

```
docker build -t lab13-app:1.0 .
docker tag lab13-app:1.0 <registry>/lab13-app:1.0
docker push <registry>/lab13-app:1.0
```

2. Create the smallest cluster on two providers (EKS, AKS, GKE). Time how long creation takes. Check current control-plane pricing first — it varies.

```
eksctl create cluster --name lab13 --nodes 2 --node-type t3.medium
az aks create -g rg-lab13 -n aks-lab13 --node-count 2 --attach-acr <acr-name> --
generate-ssh-keys
az aks get-credentials -g rg-lab13 -n aks-lab13
gcloud container clusters create-auto lab13 --region=<region>
gcloud container clusters get-credentials lab13 --region=<region>
```

3. Deploy with one manifest, changing only the image path. Expose it with a Service of type LoadBalancer.

```
kubectl apply -f deployment.yaml
kubectl get svc lab13-app -w
```

4. Scale to five replicas and confirm the nodes accommodate them.

```
kubectl scale deployment lab13-app --replicas=5
kubectl get pods -o wide
```

5. Staged failure: deploy a wrong image tag. Diagnose the ImagePullBackOff:

```
kubectl describe pod <pod-name>
```

6. Record what the managed service handled for you and what you still configured yourself.

VERIFY

- [ ] App reachable through a cloud load balancer on both providers
- [ ] ImagePullBackOff cause read from pod events

TEARDOWN

- [ ] Delete the Service first (it removes the cloud load balancer), then the cluster
- [ ] Delete registry images and repositories
- [ ] This is the most expensive lab so far — confirm in billing tomorrow

IF IT GOES WRONG

- ImagePullBackOff: wrong tag, wrong registry path, or the cluster lacks permission to pull from the registry.
- Service stuck on “pending” external IP: the cluster can’t create a load balancer — check quotas and the cluster’s identity permissions.
- Pods stuck “Pending”: not enough node capacity; check kubectl describe for the scheduler message.

Reflect

1. What remained your responsibility on the managed service?
2. Which provider felt most “finished” out of the box?

LAB 14 · Video 14

Event-Triggered Functions

TIME	EST. COST	PROVIDERS
2 h	$0	AWS · Azure · GCP

Objective. Deploy a function triggered by a storage event on each provider and trace one execution end to end.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- A bucket per provider
- Basic Python or JavaScript

TASKS

1. Write a function that logs the name and size of an uploaded object.
2. Deploy it on each provider with a storage-upload trigger: AWS Lambda with an S3 event, Azure Functions with a Blob trigger, GCP Cloud Functions with a Cloud Storage trigger.

```
aws lambda create-function --function-name lab14 --runtime python3.12 --handler
app.handler \
  --zip-file fileb://app.zip --role <execution-role-arn>
func azure functionapp publish <function-app-name>
gcloud functions deploy lab14 --gen2 --runtime=python312 --region=<region> \
  --entry-point=handler --trigger-bucket=lab14-<initials>
```

3. Upload a file and find the log line in CloudWatch Logs, Application Insights, and Cloud Logging.

```
aws logs tail /aws/lambda/lab14 --follow
gcloud functions logs read lab14 --region=<region>
```

4. Trace the execution: upload time, trigger time, start, end. Note the gap.
5. Wait 20 minutes, upload again (cold), then upload three more quickly (warm). Compare durations.
6. Set a timeout and a maximum concurrency or instance limit on each function — your protection against a runaway bill.

```
aws lambda put-function-concurrency --function-name lab14 --reserved-concurrent-
executions 5
gcloud functions deploy lab14 --max-instances=5 --timeout=30s   # plus original flags
```

VERIFY

- [ ] One upload traced end to end on each provider
- [ ] Cold and warm durations recorded
- [ ] Concurrency limits set

TEARDOWN

- [ ] Delete functions, triggers, and buckets
- [ ] Delete function log groups / Application Insights resources

IF IT GOES WRONG

- No logs at all usually means the trigger never fired — check the event wiring and that the function has permission to be invoked.
- “AccessDenied” inside the function: its execution role lacks permission, not you.
- Never let a function write to the same bucket that triggers it — that loop is the runaway bill.

Reflect

1. Where does serverless pricing beat a small always-on VM, and where does it lose?
2. What caused the cold-start gap?

LAB 15 · Video 15

An Event-Driven Flow That Survives Failure

TIME	EST. COST	PROVIDERS
90 min	$0	One provider

Objective. Build a queue-plus-function flow and confirm messages survive a consumer failure.

BEFORE YOU START

- Lab 14 (functions)

TASKS

1. Create a queue with a dead-letter queue: Amazon SQS, Azure Service Bus, or Pub/Sub with a dead-letter topic. Set max delivery attempts to 3.

```
aws sqs create-queue --queue-name lab15-dlq
aws sqs create-queue --queue-name lab15 --attributes file://redrive.json
gcloud pubsub topics create lab15
gcloud pubsub topics create lab15-dlq
gcloud pubsub subscriptions create lab15-sub --topic=lab15 \
  --dead-letter-topic=lab15-dlq --max-delivery-attempts=5
```

2. Connect a function that processes each message and logs its ID.
3. Send 20 messages and confirm all 20 are processed.

```
for i in $(seq 20); do aws sqs send-message --queue-url <queue-url> --message-body
"msg-$i"; done
```

4. Disable the consumer. Send 20 more and confirm they wait in the queue (check queue depth).

```
aws lambda update-event-source-mapping --uuid <mapping-id> --no-enabled
aws sqs get-queue-attributes --queue-url <queue-url> --attribute-
names ApproximateNumberOfMessages
```

5. Re-enable the consumer and confirm all 20 are processed.
6. Send one poison message that always throws. Confirm it lands in the dead-letter queue after three attempts.

VERIFY

- [ ] 40 of 40 good messages processed
- [ ] Queue depth rose while the consumer was off

- [ ] Poison message in the dead-letter queue

TEARDOWN

- [ ] Delete function, queue, and dead-letter queue

IF IT GOES WRONG

- Pub/Sub requires at least 5 delivery attempts before dead-lettering, and its service account needs publish rights on the dead-letter topic.
- Messages reappearing: the consumer failed or exceeded the visibility timeout before deleting them.
- Set the queue visibility timeout longer than the function timeout.

Reflect

1. What would have happened without a queue between producer and consumer?
2. How would you alert on a growing dead-letter queue?

LAB 16 · Video 16

Query a Data Warehouse and Watch the Cost

TIME	EST. COST	PROVIDERS
90 min	< $1	One provider

Objective. Load a dataset into a cloud data warehouse, run analytical queries, and see what each query costs.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- Basic SQL (SELECT, WHERE, GROUP BY)

TASKS

1. Use BigQuery (sandbox or free tier) unless you have reason to use Redshift Serverless or Synapse serverless — check their minimum charges first.
2. Set a safety cap before querying: a maximum bytes billed per query, or a workgroup / query limit.

   ```
   bq query --use_legacy_sql=false --maximum_bytes_billed=1000000000 '<your SQL>'
   ```

3. Load a public dataset (or query one in place) of at least a few GB.
4. Run three analytical queries. For each, record bytes scanned and estimated cost.

   ```
   bq query --use_legacy_sql=false --dry_run \
     'SELECT name, SUM(number) AS total FROM `bigquery-public-
   data.usa_names.usa_1910_current` GROUP BY name'
   ```

5. Compare SELECT * with selecting only the columns you need. Then query a partitioned table with and without a partition filter.
6. Run one query that exceeds your cap and confirm it is refused.

VERIFY

- ☐ Bytes scanned recorded for every query
- ☐ Column selection and partition filter both reduce cost
- ☐ Cap blocks the oversized query

TEARDOWN

- [ ] Delete datasets / tables you created

```
IF IT GOES WRONG
```

- Use --dry_run (or the console estimate) to see bytes scanned before running anything.
- LIMIT does not reduce bytes scanned in BigQuery — only selecting fewer columns or partitions does.
- Load failures usually come from schema mismatches; check the job error for the row and column.

Reflect

1. When would you need a warehouse rather than the database from Lab 10?
2. What habit would have prevented the most expensive query you ran?

LAB 17 · Video 17

Compare Pre-Built AI Services

TIME	EST. COST	PROVIDERS
90 min	< $1	AWS · Azure · GCP

Objective. Call equivalent AI services on each provider with the same input and compare results, latency, and price.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- APIs enabled for each AI service
- A test image and paragraph

TASKS

1. Pick one image and one paragraph of text. Use the same inputs everywhere.
2. Image: call Amazon Rekognition, Azure AI Vision, and Cloud Vision for labels. Record top labels and confidence.

```
aws rekognition detect-labels --image-bytes fileb://photo.jpg --max-labels 10
gcloud ml vision detect-labels photo.jpg
```

3. Text: call Amazon Comprehend, Azure AI Language, and Cloud Natural Language for sentiment and entities.

```
aws comprehend detect-sentiment --language-code en --text "<paragraph>"
gcloud ml language analyze-sentiment --content="<paragraph>"
curl -X POST "<endpoint>/language/:analyze-text?api-version=2023-04-01" \
  -H "Ocp-Apim-Subscription-Key: <key>" -H "Content-Type: application/json" -
d @sentiment.json
```

4. Optional: send the same prompt to a hosted model through Amazon Bedrock, Azure OpenAI, or Vertex AI. Note the model, tokens, and cost.
5. Record latency and unit price for each call. Compute cost per 1,000 calls.
6. Note anything about data handling: region, retention, whether inputs are used for training.

VERIFY

- [ ] Same inputs used for all calls
- [ ] Cost per 1,000 calls calculated for each

TEARDOWN

- [ ] Delete API keys and any endpoints or deployments you created

IF IT GOES WRONG

- “API not enabled” on GCP: enable it with gcloud services enable vision.googleapis.com.
- Generative AI models often need access approval or regional availability — request it early.
- Poor results? Check image size, format, and language code before blaming the model.

Reflect

1. Which provider was best on your input, and was it worth the price difference?
2. What data would you never send to these APIs?

LAB 18 · Video 18

Measure the Edge

TIME	EST. COST	PROVIDERS
90 min	< $1	One provider

Objective. Serve content from an edge network and measure the latency improvement from different locations.

BEFORE YOU START

- Lab 7 (buckets)

TASKS

1. Host a small static site in a bucket in your home region.
2. Put a CDN in front of it: Amazon CloudFront, Azure Front Door, or Cloud CDN. Set cache headers on the files.

   ```
   aws s3 cp ./site s3://lab18-<initials>/ --recursive --cache-control "max-age=3600"
   ```

3. Measure time to first byte against origin and edge:

   ```
   curl -o /dev/null -s -w "%{time_starttransfer}\n" https://<url>/index.html
   ```

4. Repeat from at least three locations — a cloud VM or shell in distant regions, or an online multi-location test.
5. Change a file and observe stale content. Invalidate the cache and confirm the new version.

   ```
   aws cloudfront create-invalidation --distribution-id <id> --paths "/*"
   az afd endpoint purge -g rg-lab18 --profile-name afd-lab18 --endpoint-name <endpoint>
   --content-paths "/*"
   gcloud compute url-maps invalidate-cdn-cache <url-map> --path "/*"
   ```

VERIFY

- ☐ Origin vs edge TTFB recorded from three locations
- ☐ Invalidation serves the updated file

TEARDOWN

- ☐ Disable and delete the CDN distribution / profile (can take several minutes)
- ☐ Delete the bucket

IF IT GOES WRONG

- Seeing the old file? That’s the cache working. Invalidate or use versioned file names.
- 403 from the CDN usually means it lacks permission to read the private origin bucket.
- CDN changes take minutes to propagate — wait before re-testing.

Reflect

1. Which locations benefited most, and why?
2. What content should never be cached?

LAB 19 · Video 19

Migrate a Workload

TIME	EST. COST	PROVIDERS
3 h	$1–5	One provider

Objective. Assess, move, and verify a small workload in the cloud, then compare its running cost with the original.

BEFORE YOU START

- Labs 6, 8, and 10
- A small workload to move

TASKS

1. Pick a workload: an app and database from your home lab, or a two-container sample (web + database).
2. Assess it: components, ports, dependencies, data size, and acceptable downtime.

```
ss -tulpn                       # listening ports
du -sh /var/lib/postgresql      # data size
```

3. Choose a strategy per component (rehost, replatform, refactor, retain, retire) and justify each.
4. Rehost the app on a VM. Replatform the database onto a managed service, using the provider's migration tool or a dump-and-restore.

```
pg_dump -Fc -h <old-host> -U <user> appdb > appdb.dump
pg_restore -h <managed-endpoint> -U <user> -d appdb --no-owner appdb.dump
```

5. Cut over and run a test checklist: pages load, writes persist, scheduled jobs run.
6. Estimate monthly cloud cost with the pricing calculator and compare with on-premises (hardware amortization, power, your time).

VERIFY

- [ ] Test checklist passes after cut-over
- [ ] Every dependency from the assessment is accounted for

TEARDOWN

- [ ] Delete the VM, managed database, and any migration service resources

IF IT GOES WRONG

- Works locally, broken in the cloud: check hard-coded hostnames, file paths, and firewall ports.
- Database connection fails after migration: the app is still pointed at the old host or lacks SSL.
- A cost that balloons post-migration is often an oversized VM chosen “to be safe.”

Reflect

1. Which dependency did your assessment miss?
2. Would you replatform or refactor this workload next?

LAB 20 · Video 20

Design a Well-Architected System

TIME	EST. COST	PROVIDERS
2 h	$0	Provider of choice

Objective. Design a resilient, scalable architecture and justify each choice against the framework pillars.

SCENARIO

A ticket-booking site with 5,000 daily users, traffic spikes of 20× when popular events go on sale, payments through a third-party processor, and a small operations team.

BEFORE YOU START

- Videos 6–20
- A diagramming tool or paper

TASKS

1. Draw the architecture: entry, compute, data, async work, and observability.
2. For each component, write one sentence per relevant pillar: operational excellence, security, reliability, performance, cost.
3. Mark every single point of failure. Remove each or document why you accept it.
4. Show how the design handles a 20× spike and how it scales back down.
5. Estimate monthly cost at normal load and at peak with the pricing calculator.

VERIFY

- [ ] No unexplained single points of failure
- [ ] Every service choice has a pillar-based justification

IF IT GOES WRONG

- If one box fails and the whole site goes down, that box is a single point of failure.
- Check every arrow: is it synchronous? Could a queue absorb the spike instead?

Reflect

1. Which pillar did you trade off, and why?
2. What would change at 10× the users?

LAB 21 · Video 21

Survive the Loss of a Zone

TIME	EST. COST	PROVIDERS
2.5 h	$1–5	One provider

Objective. Deploy an application across availability zones and demonstrate it survives the loss of one.

BEFORE YOU START

- Labs 6 and 9

TASKS

1. Create an auto-scaling group / scale set / managed instance group spanning at least two zones, behind a load balancer.

```
aws autoscaling create-auto-scaling-group --auto-scaling-group-name asg-lab21 \
  --launch-template LaunchTemplateName=lt-lab21 --min-size 2 --max-size 4 \
  --vpc-zone-identifier "<subnet-zone-a>,<subnet-zone-b>" --target-group-arns <tg-arn>
az vmss create -g rg-lab21 -n vmss-lab21 --image Ubuntu2204 --instance-count 2 --zones
1 2
gcloud compute instance-groups managed create mig-lab21 --template=it-lab21 --size=2 -
-region=<region>
```

2. Start a monitoring loop from your machine:

```
while true; do echo "$(date +%T) $(curl -s -o /dev/null -w %{http_code} http://<lb-
address>/)"; sleep 1; done | tee lab21.log
```

3. Simulate the loss of one zone: terminate all instances in it and remove it from the group's zone list.

```
aws autoscaling update-auto-scaling-group --auto-scaling-group-name asg-lab21 --vpc-
zone-identifier "<subnet-zone-b>"
```

4. Watch the log. Record the first error, last error, and when capacity was restored in the remaining zone.
5. Compute the observed RTO and compare with what the design promised.

VERIFY

- [ ] Service kept responding (or recovered) with one zone removed

- [ ] Observed RTO calculated from the log

TEARDOWN

- [ ] Delete the scaling group, template, and load balancer

```
IF IT GOES WRONG
```

- Errors during zone loss but no recovery: the group may not have subnets in another zone.
- Some errors during failover are normal — the question is how many and for how long.

Reflect

1. How does the observed RTO compare with what you'd accept for this app?
2. What would you need to survive a regional loss?

LAB 22 · Video 22

Security Posture Assessment

TIME	EST. COST	PROVIDERS
2 h	< $1	AWS · Azure · GCP

Objective. Run a posture assessment on each account, find misconfigurations, and remediate them.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- Resources from earlier labs (or deploy a few)

TASKS

1. Enable AWS Security Hub (foundational best practices), the free tier of Microsoft Defender for Cloud, and Security Command Center standard tier.

```
aws securityhub enable-security-hub --enable-default-standards
gcloud scc findings list projects/<project-id> --filter='state="ACTIVE"'
```

2. Wait for findings to populate. Export or list the top findings by severity.

```
aws securityhub get-findings --max-items 20 \
  --filters '{"SeverityLabel":[{"Value":"CRITICAL","Comparison":"EQUALS"}]}'
az security assessment list -o table
```

3. Triage: for each high or critical finding, decide fix, accept, or not applicable.
4. Remediate at least five. Common ones: MFA missing, public storage, SSH open to 0.0.0.0/0, unused access keys, default encryption off.

```
aws ec2 revoke-security-group-ingress --group-id <sg-id> --protocol tcp --port 22 --
cidr 0.0.0.0/0
aws iam update-access-key --user-name <user> --access-key-id <key-id> --
status Inactive
```

5. Re-run or wait for re-evaluation and confirm the findings clear.

VERIFY

- [ ] Five or more findings resolved
- [ ] Every remaining high finding has a recorded decision

TEARDOWN

- [ ] Disable any paid plans or trials you enabled for this lab, or note their end dates

IF IT GOES WRONG

- Findings can take up to 24 hours to appear or clear.
- Some findings are expected in a lab (e.g. no multi-region trail). Record “accepted” with a reason rather than ignoring them.

Reflect

1. Which finding would be most dangerous in production?
2. Which fix could you automate?

LAB 23 · Video 23

Detect and Investigate Suspicious Activity

TIME	EST. COST	PROVIDERS
2 h	< $1	One provider

Objective. Enable threat detection, simulate suspicious activity in your own lab account, and investigate it through the audit trail.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- Lab 12 (audit trails)
- Lab account only

TASKS

1. Enable Amazon GuardDuty, a Defender for Cloud plan, or Security Command Center threat detection. Note any trial end date.

```
aws guardduty create-detector --enable
```

2. Generate sample findings to learn the format:

```
aws guardduty create-sample-findings --detector-id <id> --finding-types <type>
```

3. Simulate a compromised credential, in your lab account only: create an access key for a lab user, use it from a different network, attempt denied actions in a loop, and try to create a new user.

```
aws iam create-access-key --user-name lab23-user
for i in $(seq 20); do aws iam list-users --profile lab23-suspect; done
```

4. Investigate in the audit trail: which identity, from which IP, which actions, which succeeded.

```
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=AccessKeyId,AttributeValue=<key-id> --max-results 50
```

5. Contain: deactivate the key, review what it created, delete it.

```
aws iam update-access-key --user-name lab23-user --access-key-id <key-id> --
status Inactive
```

6. Write an incident timeline from first activity to containment.

VERIFY

- [ ] Simulated activity found in the audit trail
- [ ] Key deactivated and anything it created removed

TEARDOWN

- [ ] Delete the lab user and its keys
- [ ] Disable threat detection or note the trial end date

IF IT GOES WRONG

- Threat detection findings can take 15+ minutes to appear — start the simulation early.
- Can’t find events? Confirm the region; management events are recorded in the region where the call was made.

Reflect

1. How long from first suspicious action to detection?
2. What would an attacker have done next?

LAB 24 · Video 24

Find and Remove Waste

TIME	EST. COST	PROVIDERS
2 h	$0	AWS · Azure · GCP

Objective. Analyze spend on your accounts, find waste, implement optimizations, and measure the savings.

BEFORE YOU START

- At least two weeks of usage across your accounts

TASKS

1. Open AWS Cost Explorer, Azure Cost Management, and GCP billing reports. Group by service, then by the lab tag.

```
aws ce get-cost-and-usage --time-period Start=<yyyy-mm-01>,End=<yyyy-mm-dd> \
  --granularity MONTHLY --metrics UnblendedCost --group-by Type=DIMENSION,Key=SERVICE
az consumption usage list --top 20 -o table
```

2. List your top five cost drivers across all three.
3. Hunt for waste: unattached disks, old snapshots, unused public IPs, idle load balancers, stopped VMs with disks, untagged resources.

```
aws ec2 describe-volumes --filters Name=status,Values=available
aws ec2 describe-addresses --query "Addresses[?AssociationId==null]"
az disk list --query "[?diskState=='Unattached']" -o table
gcloud compute disks list --filter="-users:*"
```

4. Review recommendations from AWS Trusted Advisor / Compute Optimizer, Azure Advisor, and GCP Recommender.

```
az advisor recommendation list --category Cost -o table
gcloud recommender recommendations list --project=<project> --location=<zone> \
  --recommender=google.compute.instance.MachineTypeRecommender
```

5. Fix what you find and record the projected monthly saving of each fix.

VERIFY

- [ ] Top five drivers explained
- [ ] Every untagged resource identified and tagged or deleted

TEARDOWN

- [ ] Delete the waste you found

IF IT GOES WRONG

- Cost data lags by up to a day — today's spend won't show yet.
- Untagged costs often come from resources created by other services (snapshots, log storage, data transfer).

Reflect

1. Which waste pattern did you find most often?
2. What one habit would have avoided most of it?

LAB 25 · Video 25

Deploy from a Commit

TIME	EST. COST	PROVIDERS
3 h	< $1	One provider

Objective. Build a pipeline that deploys infrastructure and an application to the cloud from a commit.

BEFORE YOU START

- Lab 11 code in a Git repository
- A GitHub account (or provider CI)

TASKS

1. Create a repository with your app and the Terraform / OpenTofu code from Lab 11.
2. Set up CI/CD with GitHub Actions or the provider's service (CodePipeline / CodeBuild, Azure Pipelines, Cloud Build).
3. Authenticate the pipeline with workload identity federation (OIDC), not long-lived keys.

```
# .github/workflows/deploy.yml (excerpt)
permissions:
  id-token: write
  contents: read
steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: <role-arn>
      aws-region: <region>
```

4. Stages: lint and test → infrastructure plan → apply on main → deploy app → smoke test.

```
terraform fmt -check && terraform validate
terraform plan -out tfplan
terraform apply -auto-approve tfplan
```

5. Commit a visible change and watch it reach production.
6. Commit a failing test and confirm the pipeline stops before deploying.

VERIFY

- [ ] A commit reaches the running app with no manual steps

- [ ] No cloud secrets stored in the repository
- [ ] A failing test blocks deployment

TEARDOWN

- [ ] Run destroy through the pipeline or locally
- [ ] Remove the federation trust and pipeline role

```
IF IT GOES WRONG
```

- “Not authorized to perform sts:AssumeRoleWithWebIdentity”: the role’s trust policy doesn’t match your repo or branch.
- Pipeline passes but nothing changed: check it’s deploying the new image tag, not latest from cache.

Reflect

1. What did you remove from human hands?
2. What should still need a human approval?

LAB 26 · Video 26

Guardrails at Organization Scale

TIME	EST. COST	PROVIDERS
2 h	$0	One or more

Objective. Set up an organizational structure with a policy guardrail and confirm it blocks a deliberately non-compliant action.

BEFORE YOU START

- Lab 3 accounts, budgets, and CLIs
- Lab 4 (IAM)

TASKS

1. AWS: create an Organization, a Sandbox OU, and a member account. Attach a service control policy that denies actions outside your allowed regions.

```
aws organizations create-organization --feature-set ALL
aws organizations create-organizational-unit --parent-id <root-id> --name Sandbox
aws organizations create-policy --name deny-regions --type SERVICE_CONTROL_POLICY \
  --description "lab26" --content file://deny-regions.json
aws organizations attach-policy --policy-id <policy-id> --target-id <ou-id>
```

2. Azure: create a management group, move your subscription under it, and assign the built-in “Allowed locations” policy.

```
az account management-group create -n mg-lab26
az policy assignment create -n allowed-locations --policy e56962a6-4747-49cd-b67b-
bf8b01975c4c \
  --scope /providers/Microsoft.Management/managementGroups/mg-lab26 \
  --params '{"listOfAllowedLocations":{"value":["<region>"]}}'
```

3. GCP: if you have an organization (Cloud Identity free works), create a folder and set the resource-locations organization policy.

```
gcloud resource-manager org-policies allow gcp.resourceLocations in:us-locations --
folder=<folder-id>
```

4. Attempt to create a resource in a disallowed region. Capture the denial message.
5. Attempt something legitimate the guardrail should allow, to confirm it doesn’t over-block.

VERIFY

- [ ] Non-compliant action blocked with a policy-attributed error
- [ ] Compliant action still succeeds

TEARDOWN

- [ ] Remove policies before closing member accounts
- [ ] Close or suspend any member accounts you created

IF IT GOES WRONG

- SCPs don’t apply to the management account itself — test from a member account.
- Azure Policy assignments can take up to 30 minutes to take effect.

Reflect

1. What legitimate work could this guardrail block?
2. Which guardrail would you add next?

LAB 27 · Video 27

One App, Three Clouds

TIME	EST. COST	PROVIDERS
3–4 h	$5–15	AWS · Azure · GCP

Objective. Deploy the same containerized application across all three clouds using provider-neutral tooling and compare the effort.

BEFORE YOU START

- Labs 11 and 13
- Container image from Lab 13

TASKS

1. Reuse the image from Lab 13. Push it to all three registries.
2. Deploy it with one Terraform / OpenTofu project with a module per provider. Serverless containers (AWS App Runner or ECS Fargate, Azure Container Apps, Cloud Run) are cheapest; managed Kubernetes is the alternative.

```
gcloud run deploy lab27 --image=<artifact-registry-image> --region=<region> --
allow-unauthenticated
az containerapp up -n lab27 -g rg-lab27 --image <acr-image> --ingress external --
target-port 8080
# AWS: define App Runner or ECS Fargate in Terraform (aws_apprunner_service)
```

3. Record: time to first successful deploy, provider-specific lines of code, and concepts that changed names or behavior.
4. Fetch the app from each URL and confirm identical responses.
5. Estimate the monthly cost of running it on all three versus one.

VERIFY

- [ ] App running on all three from the same image
- [ ] Effort metrics recorded for each

TEARDOWN

- [ ] terraform destroy for all three
- [ ] Delete registry images

IF IT GOES WRONG

- Container won’t start: check it listens on the port the platform expects (often 8080, set via PORT).
- Deploy succeeds but URL returns 403: the service isn’t set to allow unauthenticated access.

Reflect

1. What ported cleanly, and what didn’t?
2. When would you recommend multi-cloud to an employer?

LAB 28 · Video 28

Choose a Provider

TIME	EST. COST	PROVIDERS
90 min	$0	Provider-neutral

Objective. Build a weighted decision matrix for a scenario and defend your recommendation.

SCENARIO

A mid-size retailer already on Microsoft 365, wanting to analyze several years of sales data and pilot demand forecasting, with a four-person team strongest in Linux and containers.

BEFORE YOU START

- Notes from your earlier labs

TASKS

1. List 6–8 criteria that matter to this organization (existing investments, team skills, specific services, pricing, support, compliance).
2. Weight them to total 100.
3. Score each provider 1–5 on each criterion, citing evidence from your labs.
4. Run a sensitivity check: change the top weight by ±10 and see whether the winner changes.
5. Note how you would limit lock-in for the chosen provider.

VERIFY

- [ ] Every score cites evidence
- [ ] The sensitivity check is documented

IF IT GOES WRONG

- If one criterion decides everything, check whether its weight is honest.

Reflect

1. Which criterion was hardest to score objectively?
2. What would make you change your recommendation?

LAB 29 · Video 29

Root-Cause a Multi-Layer Fault

TIME	EST. COST	PROVIDERS
2 h	< $1	One provider

Objective. Apply the full troubleshooting framework to a fault with more than one cause and document your reasoning to root cause.

SCENARIO

A function that writes uploaded files to a bucket stops working after a “small cleanup” change. Stage it with a partner, or stage it yourself and wait a day: remove one permission, set a wrong region in an environment variable, and add a bucket policy deny.

BEFORE YOU START

- Labs 5, 12, and 14
- A partner to stage faults (optional)

TASKS

1. Start a fresh Appendix D worksheet.
2. Walk the diagnostic map in order: identity → configuration → network → service → provider.
3. Check the status page first and record the result.
4. For each hypothesis, record the evidence (log line, metric, audit event), the single change tested, and the outcome.

```
aws logs filter-log-events --log-group-name /aws/lambda/<function> --filter-
pattern "AccessDenied"
aws lambda get-function-configuration --function-name <function> --query Environment
```

5. Keep going after the first fix — confirm the system works end to end.
6. Turn what you learned into version 1 of your personal troubleshooting checklist.

VERIFY

- ☐ All three staged causes found
- ☐ Each found from evidence, with one change per test

TEARDOWN

- [ ] Delete the function, bucket, and role

IF IT GOES WRONG

- Fixing one cause and seeing a different error is progress — the next layer is now visible.
- Read the full error message; access-denied errors often name the exact permission and resource.

Reflect

1. Which layer took longest, and why?
2. What’s first on your personal checklist now?

LAB 30 · Video 30

Access, Connectivity, and Configuration Failures

TIME	EST. COST	PROVIDERS
3 h	$1–5	Two providers

Objective. Diagnose staged access, connectivity, and configuration failures on two providers and identify each root cause.

BEFORE YOU START

- Labs 4, 8, 9, and 11

TASKS

1. Stage three faults on each provider (a partner, or a script you wrote and don't re-read): a missing permission, a removed firewall rule or route, and a resource changed outside of code.
2. Access: trace with the IAM Policy Simulator, Azure "Check access", or GCP Policy Troubleshooter.

```
aws iam simulate-principal-policy --policy-source-arn <role-arn> \
  --action-names s3:PutObject --resource-arns "arn:aws:s3:::<bucket>/*"
gcloud policy-intelligence troubleshoot-policy iam <full-resource-name> \
  --principal-email=<email> --permission=storage.objects.create
```

3. Connectivity: trace with Reachability Analyzer, Network Watcher, or Connectivity Tests.

```
aws ec2 create-network-insights-path --source <instance-a> --destination <instance-
b> \
  --protocol tcp --destination-port 80
aws ec2 start-network-insights-analysis --network-insights-path-id <path-id>
```

4. Configuration: detect with terraform plan, then reconcile code and reality.

```
terraform plan -detailed-exitcode   # exit code 2 = drift
```

5. Confirm each fix with the original failing test.
6. Note how differently the two providers surfaced the same failure.

VERIFY

- [ ] Six faults diagnosed with the provider tool named
- [ ] Each fix confirmed by re-running the failing action

TEARDOWN

- [ ] Destroy everything created for the lab and run the Appendix C audit

IF IT GOES WRONG

- Same symptom, different cause: a timeout may be a firewall, a route, or a stopped service. Test each layer in order.
- Drift that keeps coming back means someone or something is still changing the resource.

Reflect

1. Which provider’s tooling found the cause fastest?
2. What would prevent each fault from recurring?

LAB 31 · Video 31

Performance and Cost Anomalies

TIME	EST. COST	PROVIDERS
2.5 h	$1–5	One provider

Objective. Diagnose a performance problem and a cost anomaly, remediate both, and measure the result.

BEFORE YOU START

- Labs 6, 12, and 24
- A load-testing tool (hey or similar)

TASKS

1. Deploy a small web app on an undersized VM with a small disk.
2. Load test and record p50 and p95 latency:

```
hey -z 2m -c 50 http://<address>/
```

3. Locate the bottleneck from metrics: CPU, memory, disk IOPS, network, or database. Change one thing and re-test.

```
top -b -n1 | head -15
iostat -x 5 3
free -h
```

4. Stage a cost anomaly: a scheduled function firing every few seconds, or a larger VM left running for an hour.
5. Find it the next day through cost anomaly detection or daily cost grouped by service and tag. Remediate.

```
aws ce get-anomalies --date-interval StartDate=<yyyy-mm-dd>
aws ce get-cost-and-usage --time-period Start=<yyyy-mm-dd>,End=<yyyy-mm-dd> \
  --granularity DAILY --metrics UnblendedCost --group-by Type=DIMENSION,Key=SERVICE
```

6. Decide whether the performance fix cost more money — and whether that was the right call.

VERIFY

- [ ] Before/after latency recorded with the change identified
- [ ] Cost anomaly located by tooling, not memory

TEARDOWN

- [ ] Delete the VM, function, and schedule

IF IT GOES WRONG

- Burstable instances (t3, B-series, e2-micro) slow down sharply once CPU credits run out — a classic “it got slow” cause.
- Cost anomaly detection needs history; if it’s quiet, use daily cost by service instead.

Reflect

1. Was spending more the right fix for the performance problem?
2. How would you catch the cost anomaly the same day?

LAB 32 · Video 32

Your Development Plan

TIME	EST. COST	PROVIDERS
90 min	$0	Provider-neutral

Objective. Assess your skills against the whole course, pick a specialization and lead provider, and write a 90-day plan.

BEFORE YOU START

- Your notes from all labs

TASKS

1. Rate yourself 1–5 on each course area: compute, storage, networking, identity, databases, IaC, observability, containers, serverless, security, cost, troubleshooting.
2. Choose a specialization: architecture, security, data engineering, DevOps / platform, or machine learning.
3. Choose a lead provider and one certification to target.
4. Write a 90-day plan in three 30-day blocks, each with one project and one measurable outcome.
5. Set a monthly home-lab budget and adjust your Lab 3 alarms to match.
6. Run the Appendix C audit on all three accounts one final time.

VERIFY

- [ ] Plan names a specialization, provider, and certification
- [ ] Each 30-day block has a measurable outcome
- [ ] All three accounts audited clean

TEARDOWN

- [ ] Delete everything left from earlier labs

IF IT GOES WRONG

- Pick one provider to go deep on — breadth without depth is hard to show in an interview.

Reflect

1. Which lab are you most proud of, and why?
2. What will you build in the first 30 days?

A · Naming and tagging convention

Name every resource `<type>-lab<NN>-<purpose>[-<n>]`, lowercase, hyphens only. Bucket names must be globally unique, so append your initials: `lab07-jd-logs`.

Tag / label key	Example	Purpose
`course`	`cloud3`	Filters all course resources in cost reports
`lab`	`lab08`	Which lab created it
`owner`	`jdoe`	Who to ask; required for shared accounts
`expires`	`2026-10-15`	Anything past this date is safe to delete

GCP labels must be lowercase. Azure tags apply per resource or resource group — put each lab in its own resource group (`rg-lab08`) so one delete removes everything.

B · Three-cloud service map

One concept, three vocabularies.

Concept	AWS	Azure	GCP
Virtual machines	EC2	Virtual Machines	Compute Engine
Auto-scaling	Auto Scaling groups	VM Scale Sets	Managed instance groups
Object storage	S3	Blob Storage	Cloud Storage
Block storage	EBS	Managed Disks	Persistent Disk
File storage	EFS / FSx	Azure Files	Filestore
Virtual network	VPC	Virtual Network	VPC
Load balancing	Elastic Load Balancing	Load Balancer / App Gateway	Cloud Load Balancing
DNS	Route 53	Azure DNS	Cloud DNS
CDN	CloudFront	Front Door	Cloud CDN
Identity	IAM / IAM Identity Center	Entra ID + RBAC	Cloud IAM
Relational DB	RDS / Aurora	Azure SQL / Database for PostgreSQL	Cloud SQL / AlloyDB
NoSQL	DynamoDB	Cosmos DB	Firestore / Bigtable
IaC (native)	CloudFormation	ARM / Bicep	Deployment Manager / Infrastructure Manager
Monitoring	CloudWatch	Azure Monitor	Cloud Monitoring
Audit trail	CloudTrail	Activity Log	Cloud Audit Logs
Container registry	ECR	Container Registry	Artifact Registry
Managed Kubernetes	EKS	AKS	GKE
Serverless containers	App Runner / Fargate	Container Apps	Cloud Run
Functions	Lambda	Azure Functions	Cloud Functions
Queue / pub-sub	SQS / SNS / EventBridge	Service Bus / Event Grid	Pub/Sub / Eventarc
Data warehouse	Redshift	Synapse / Fabric	BigQuery
ML platform	SageMaker	Azure Machine Learning	Vertex AI

Concept	AWS	Azure	GCP
Generative AI	Bedrock	Azure OpenAI	Vertex AI (Gemini)
Key management	KMS	Key Vault	Cloud KMS
Threat detection	GuardDuty	Defender for Cloud	Security Command Center
Org hierarchy	Organizations / OUs	Management Groups	Organization / Folders
Guardrails	Service control policies	Azure Policy	Organization policies
Cost tools	Cost Explorer / Budgets	Cost Management	Billing reports / Budgets

C · End-of-session teardown audit

Run before closing every session. Anything that returns a result you didn't intend to keep gets deleted now.

AWS — everything tagged for the course, in every region

```
aws resourcegroupstaggingapi get-resources --tag-filters Key=course,Values=cloud3 \
  --query "ResourceTagMappingList[].ResourceARN"
for r in $(aws ec2 describe-regions --query "Regions[].RegionName" --output text); do
  aws ec2 describe-instances --region $r --query "Reservations[].Instances[].InstanceId" -
-output text
done
```

Azure — every resource group and what is in it

```
az group list -o table
az resource list --tag course=cloud3 -o table
```

GCP — every asset in the lab project

```
gcloud asset search-all-resources --scope=projects/<project-id> \
  --format="table(assetType,name)"
```

Commonly forgotten: unattached disks and snapshots, reserved public IPs, NAT gateways, load balancers, log workspaces, container registries, and resources in a region other than your default. Check the billing page the next morning.

D · Troubleshooting checklist

Copy this for every fault in Labs 5, 29, 30, and 31. Fill it in as you go, not afterwards.

1 · Problem statement

Who is affected, what they see, since when.

2 · Recent changes

Deploys, config edits, policy changes in the last 24 hours.

3 · Provider status

Status page checked at (time). Incident? Region?

4 · Layer and owner

Identity / configuration / network / service / provider — mine or theirs?

5 · Evidence

Log lines, metrics, audit events — with source and timestamp.

6 · Change tested

One change. What you expected; what happened.

7 · Root cause

The cause, and why the evidence supports it.

8 · Cost

Of the fault, of the fix, and of anything left running.